

Phân phối, dịch chuyển và ngoài phân phối (Distribution Shift and OOD)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (giả định IID và vì sao mọi bảo đảm học máy đứng trên nó); B/05-thiet-ke-bai-toan (thiên lệch chọn mẫu (*selection bias*), khung lấy mẫu); B/07-chia-du-lieu-va-danh-gia (chương này gỡ bỏ đúng giả định mà giao thức ở đó dựa vào).

TÓM TẮT

Chương này nói về chuyện xảy ra khi dữ liệu thật không còn cùng phân phối với dữ liệu huấn luyện — tức là gần như luôn luôn. Đọc xong bạn thay được câu “bị domain shift” bằng một chẩn đoán cụ thể chỉ ra nên gán nhãn lại, nên chỉnh đầu ra, hay nên thu thập thêm; bạn dựng được một quy trình giám sát trôi phân phối; và bạn biết vì sao mô hình lại mong manh đến vậy. Nếu chỉ nhớ một điều: ba loại dịch chuyển — $p(x)$ đổi, $p(y)$ đổi, $p(y | x)$ đổi — cho cùng một triệu chứng nhưng đòi ba cách chữa loại trừ nhau, và chỉ loại thứ ba mới bắt buộc phải gán nhãn lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phân phối xác suất và ký hiệu** $p(\cdot)$ — $p(x)$ mô tả những ảnh nào hay xuất hiện và những ảnh nào hiếm; $p(y)$ mô tả tỉ lệ các lớp; $p(y | x)$ là “cho trước ảnh này, nhãn đúng của nó có khả năng là gì”; $p(x | y)$ là “ảnh của lớp này trông thế nào”. Toàn bộ chương xoay quanh đúng một câu hỏi: trong bốn thứ đó, thứ nào đã thay đổi.
- **giả định IID** — giả định rằng mỗi mẫu được rút ra một cách độc lập từ cùng một phân phối, cho cả dữ liệu huấn luyện lẫn dữ liệu tương lai. Mọi bảo đảm lý thuyết của học máy đứng trên giả định này, và chương này nói về chuyện xảy ra khi nó vỡ.
- **ba loại dịch chuyển phân phối** — *covariate shift* — $p(x)$ đổi (camera mới, ánh sáng khác) nhưng quan hệ ảnh–nhãn $p(y | x)$ vẫn đúng. *Label shift* — tỉ lệ các lớp $p(y)$ đổi trong khi ảnh của mỗi lớp vẫn y hệt. *Concept drift* — chính $p(y | x)$ đổi, tức cùng một ảnh nay phải gán nhãn khác. Ba loại cho cùng một triệu chứng nhưng đòi ba cách chữa loại trừ nhau.
- **mô hình, huấn luyện và tối thiểu hoá rủi ro thực nghiệm (empirical risk minimization, ERM)** — mô hình là hàm có tham số biến ảnh thành dự đoán; huấn luyện là tìm tham số làm trung bình mất mát trên tập huấn luyện nhỏ nhất — đúng cái nguyên tắc gọn ấy có tên là ERM. Nó là *baseline* mà mọi phương pháp phức tạp hơn trong chương này phải vượt qua.
- **tiên nghiệm (prior) và tỉ số cược (odds)** — tiên nghiệm là xác suất của một lớp *trước khi* nhìn ảnh, tức tỉ lệ lớp đó trong dữ liệu; tỉ số cược của một xác suất p là $p/(1 - p)$, một cách viết khác của cùng thông tin nhưng nhân chia được. Cần biết vì phép chữa rẻ nhất trong cả chương chỉ là nhân tỉ số cược với tỉ lệ giữa hai tiên nghiệm.
- **đuôi dài (long-tail)** — **phân biệt với mất cân bằng lớp** — mất cân bằng lớp là vài lớp có ít mẫu hơn hẳn các lớp khác. Đuôi dài là tình huống có *rất nhiều* lớp mà mỗi lớp chỉ có dăm bảy mẫu, nên vấn đề không còn là tỉ lệ mà là không đủ dữ liệu để học được lớp đó là gì. Phân biệt này quan trọng vì cách chữa của trường hợp đầu không dùng được cho trường hợp sau.
- **ngoài phân phối (OOD), softmax, độ tin cậy và hiệu chuẩn (calibration)** — softmax biến các điểm số thô ở đầu ra thành một bộ số dương cộng lại bằng 1, đọc như xác suất; giá trị lớn nhất trong đó gọi là độ tin cậy của mô hình. Hiệu chuẩn tốt nghĩa là khi mô hình nói 0,8 thì nó đúng khoảng 80% số lần. Một đầu vào *ngoài phân phối* là đầu vào khác hẳn mọi thứ mô hình từng thấy

lúc huấn luyện, và “phát hiện OOD” là cố nhận ra nó để hệ thống nói “tôi không chắc” thay vì trả lời bừa.

- **embedding, augmentation và corruption** — embedding là vector vài trăm chiều biểu diễn một ảnh, ảnh giống nhau cho vector gần nhau — dùng ở đây để so hai tập dữ liệu mà không cần nhãn. Augmentation là cố ý biến dạng ảnh *huấn luyện* (lật, đổi màu, thêm nhiễu, làm mờ) để mô hình bớt phụ thuộc chi tiết không quan trọng; corruption là cùng loại biến dạng đó nhưng áp lên tập *kiểm tra* để đo độ bền. Cùng một phép biến đổi, hai mục đích ngược nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Mô hình đạt 95% trên tập test và 60% ở khách hàng đầu tiên. Ba câu hỏi nào tách được nguyên nhân, và câu nào cho phép chữa mà không cần huấn luyện lại một tham số nào?
- Q2. Vì sao một mạng có thể phân biệt bò với lạc đà rất giỏi mà thực ra chưa bao giờ nhìn con vật?
- Q3. Nếu hàng trăm phương pháp tổng quát hoá miền đều tốt hơn ERM trong bài báo của chính chúng, vì sao khi tune công bằng thì không cái nào thắng?
- Q4. Vì sao độ tin cậy của mô hình lại là tín hiệu cảnh báo tệ nhất, đúng vào lúc bạn cần nó nhất?

1.1 Đặt vấn đề

Mọi kết quả lý thuyết ở B/04 — từ chặn tổng quát hoá tới chính khái niệm “sai số kỳ vọng” — đứng trên một giả định duy nhất: dữ liệu huấn luyện và dữ liệu tương lai được rút độc lập từ cùng một phân phối. Trong thị giác máy tính, giả định đó gần như luôn sai. Camera được thay sau mười tám tháng, firmware ISP cập nhật, mùa đổi, khách hàng mới ở một thành phố khác, một loại xe mới xuất hiện trên đường.

Sai lầm đắt nhất của người mới ở đây không phải chọn sai phương pháp thích nghi, mà là chẩn đoán quá thô. Hệ thống tụt tám điểm sau khi triển khai, và kết luận duy nhất được đưa ra là “bị domain shift” — một câu vừa đúng vừa vô dụng, vì nó không dẫn tới hành động nào. Ba nguyên nhân hoàn toàn khác nhau đều cho ra cùng triệu chứng ấy, và chúng đòi ba cách chữa loại trừ lẫn nhau: một cái cần gán nhãn lại, một cái chỉ cần vài phép nhân ở đầu ra, một cái cần thu thập thêm dữ liệu miền mới.

Có một trục giác từ robotics giúp đóng khung cả chương. Một hệ định vị được điều chỉnh cho một mô hình nhiễu cụ thể chạy tuyệt vời trong đúng điều kiện nó được điều chỉnh; đổi cảm biến hay đổi bề mặt sàn thì nó không báo lỗi, nó chỉ *sai một cách tự tin*. Mạng nơ-ron còn tệ hơn ở chỗ nó không có **phần dư** (*residual*) để bạn nhìn vào, nên nó không có cách nào tự nói rằng mình đang ở ngoài vùng hợp lệ. Phần lớn chương này là các cách xây dựng nhân tạo cái phần dư ấy.

→ **hệ quả của B/04** — giả định IID là nền của mọi bảo đảm tổng quát hoá; gỡ nó ra thì các bảo đảm đó mất hiệu lực, chứ không phải yếu đi.

TRỰC GIÁC

Hãy hình dung một robot định vị bằng cách đếm số bước từ cửa phòng thay vì đọc bản đồ. Trong toà nhà nó được hiệu chỉnh, kết quả chính xác tuyệt đối và không có cách nào phân biệt nó với một hệ định vị thật. Sang toà nhà khác — cùng loại hành lang, cùng loại cửa, chỉ khác chiều dài — nó sai hoàn toàn ngay bước đầu tiên. Nó không hỏng; nó đã học đúng thứ tương quan mạnh nhất với đáp án trong dữ liệu nó thấy. Đây chính là cơ chế của **shortcut learning**, và nó giải thích vì sao độ chính xác cao trên tập test không hề bảo đảm hệ thống hiểu bài toán.

1.2 Ba loại dịch chuyển (Covariate, Label and Concept Shift)

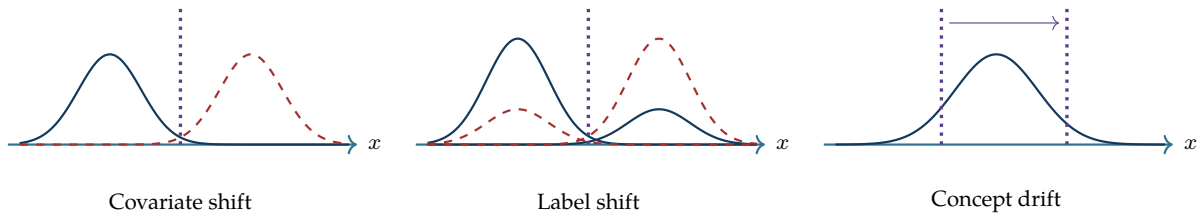
Muốn thay “bị shift” bằng một chẩn đoán dùng được, phải tách bài toán theo cấu trúc của phân phối chung $p(x, y)$. Nói gọn thì có ba cách phân rã nó, và mỗi cách cho một loại dịch chuyển với một cách chữa riêng.

A. Phân biệt bằng cấu trúc của $p(x, y)$

Bảng 1.1. Ba loại dịch chuyển. Cột cuối là lý do phải phân biệt: cách chữa của loại này vô dụng với loại kia.

Loại	Cái gì đổi	Dấu hiệu	Cách xử lý
dịch chuyển hiệp biến (<i>covariate shift</i>)	$p(x)$ đổi, $p(y x)$ giữ nguyên	Ảnh trông khác đi nhưng quan hệ ảnh–nhãn vẫn đúng	Reweighting theo mật độ, augmentation, domain adaptation
dịch chuyển nhãn (<i>label shift</i>)	$p(y)$ đổi, $p(x y)$ giữ nguyên	Tỉ lệ lớp đổi; ảnh của mỗi lớp vẫn y hệt	Hiệu chỉnh tiên nghiệm ngay ở đầu ra, không cần train lại
trôi khái niệm (<i>concept drift</i>)	$p(y x)$ đổi	Cùng một ảnh nay phải gán nhãn khác	Gán nhãn lại và huấn luyện lại; không có đường tắt

Hình 1.1 vẽ ba loại đó trên một trục đặc trưng duy nhất, và điểm đáng nhìn là chúng làm biến dạng những bộ phận khác nhau của cùng một bức tranh: hai loại đầu di chuyển *dữ liệu* quanh một biên quyết định đứng yên, còn loại thứ ba di chuyển *chính biên quyết định*. Đó là lý do hai loại đầu chữa được bằng số học trong khi loại thứ ba đòi nhãn mới.



Hình 1.1. Ba loại dịch chuyển trên một trục đặc trưng. Nét liền xanh là phân phối lúc huấn luyện, nét đứt đỏ là lúc triển khai, nét chấm tím là biên quyết định. Hai panel đầu: dữ liệu dịch chuyển, biên đứng yên. Panel cuối: dữ liệu y nguyên, biên dịch chuyển — và đó là lý do duy nhất trong ba trường hợp mà nhãn cũ trở thành nhãn sai.

B. Ví dụ tính tay: hiệu chỉnh tiên nghiệm khi có label shift

Label shift đáng có một ví dụ số vì nó là trường hợp duy nhất chữa được gần như miễn phí, và cũng là trường hợp hay bị bỏ qua nhất. Giả sử bạn huấn luyện một bộ phân loại nhị phân trên tập cân bằng (tỉ lệ dương 50%), còn khi triển khai tỉ lệ dương thật chỉ là 5%. Mô hình trả về $p = 0,60$ cho một mẫu.

- Đổi sang **tỉ số cược** (*odds*): $0,60/0,40 = 1,50$.
- Vì $p(x | y)$ không đổi, **tỉ số độ hợp lý** (*likelihood ratio*) giữ nguyên và chỉ phần tiên nghiệm cần sửa. Nhân với $\frac{0,05/0,95}{0,50/0,50} \approx 0,0526$, được 0,079.
- Đổi ngược về xác suất: $p \approx 0,079/1,079 \approx 0,073$.

Phép tính này nói điều gì? Rằng một dự đoán “khá chắc là dương” thật ra là “7% khả năng dương”, và toàn bộ sự chênh lệch đến từ việc mô hình mang theo tiên nghiệm của tập huấn

luyện chứ không phải của thế giới. Không train lại một tham số nào; chỉ cần biết tỉ lệ lớp mới. Ngược lại, nếu vấn đề thật là concept drift thì phép nhân này không những vô ích mà còn làm sai lệch thêm, vì giả định $p(x | y)$ bất biến đã vỡ ngay từ đầu.

↔ **tiền đề** E/25 — hiệu chỉnh tiên nghiệm là một thao tác vận hành: nó sửa đầu ra theo tỉ lệ lớp thật, và chỉ dùng được khi đầu ra đã hiệu chuẩn.

1.3 Biết là đã trôi, chưa biết có hại (Shift Detection)

Trước khi chữa phải phát hiện, và phát hiện rẻ hơn nhiều so với người ta tưởng. Hai công cụ thực dụng, cộng một giới hạn phải luôn nhớ:

- **Khoảng cách trên không gian *embedding*** — so phân phối đặc trưng của dữ liệu huấn luyện với dữ liệu đang chạy. Rẻ, chạy liên tục được, nhưng con số khoảng cách tự nó khó diễn giải.
- **Bộ phân loại phân biệt miền** — huấn luyện một bộ phân loại nhị phân để tách “mẫu train” khỏi “mẫu production”. Đây là dụng cụ đo tốt hơn vì kết quả đọc được ngay:
 - **AUC $\approx 0,5$** — hai tập không phân biệt được, chưa có dịch chuyển đáng kể.
 - **AUC $\approx 0,95$** — chỉ cần nhìn ảnh cũng biết nó đến từ đâu.
 - **Lợi ích kèm theo** — các đặc trưng mà bộ phân biệt dựa vào chính là mô tả cụ thể của dịch chuyển, nên nó vừa báo động vừa chẩn đoán.
- **Giới hạn chung của cả hai** — chúng trả lời “phân phối có đổi không”, không phải “việc đó có làm hỏng hệ thống không”. Hai điều đó tách rời theo cả hai chiều: đổi tông màu toàn cục là thay đổi lớn về thống kê nhưng vô hại nếu mô hình không dùng màu; ngược lại một thay đổi bé xíu đúng vào đặc trưng mô hình dựa vào lại đủ làm sập nó.

Hệ quả thực hành: tín hiệu dịch chuyển phải luôn được đối chiếu với một metric thật trên một tập nhỏ có nhãn thu ở miền đích. Giám sát dịch chuyển mà không có mẫu có nhãn nào ở miền đích là giám sát mù. Vì thế ngân sách nên đổ vào việc lấy vài trăm nhãn mỗi tháng từ production, trước khi đổ vào thuật toán thích nghi.

1.4 Thích nghi hay tổng quát hoá (Domain Adaptation vs Generalization)

Đến khi phải hành động, có một phân đôi quyết định toàn bộ chiến lược, và nó phụ thuộc vào đúng một câu hỏi: *bạn có mẫu từ miền đích hay không?*

1. Mạch 1 — từ thích nghi tới tổng quát hoá

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

có mẫu miền đích (dù không nhãn), và chỉ cần chạy tốt ở đúng miền đó.

thích nghi miền (*domain adaptation*): căn chỉnh phân phối đặc trưng giữa hai miền, hoặc tự huấn luyện bằng nhãn giả trên miền đích.

hai miền chia sẻ cùng quan hệ $p(y | x)$, tức đây là covariate shift chứ không phải concept drift.

mô hình bị chuyên biệt hoá cho một phân phối cụ thể; miền thứ ba xuất hiện thì làm lại từ đầu.

2 Vấn đề

Giải pháp

chưa từng thấy miền đích và không biết nó sẽ là gì.

tổng quát hoá miền (*domain generalization*): học biểu diễn bất biến qua nhiều miền huấn luyện, với hy vọng cái bất biến đó cũng đúng ở miền chưa thấy [1].

Giá phải trả	phải hy sinh hiệu năng trung bình để lấy độ bền — một biểu diễn chỉ giữ phần bất biến thì bỏ đi thông tin có ích ở miền hiện tại.
Giả định	tồn tại một cơ chế nhân quả bất biến và bạn có đủ miền huấn luyện đa dạng để nhận ra nó.
Vấn đề mới	giả định đó có được thoả trên dữ liệu thật không?
Hệ quả	đây là chỗ cần trung thực: một khảo sát có kiểm soát cho thấy khi mọi phương pháp tổng quát hoá miền được tune công bằng với cùng ngân sách, không phương pháp nào vượt được ERM thuần một cách nhất quán [2]. Kết quả này không nói “đừng làm DG”; nó nói rằng phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình, không từ thuật toán. Đây đúng là <i>adaptive overfitting</i> ở B/07, lần này xuất hiện ở cấp độ cả một dòng nghiên cứu.
3 Vấn đề	cái ta thật sự quan tâm là nhóm tẻ nhất, không phải trung bình.
Giải pháp	tối ưu trực tiếp cho nhóm tẻ nhất thay vì cho trung bình [3].
Giả định	bạn biết trước các nhóm là gì — nghĩa là quay lại đúng vấn đề metadata ở B/07: không có nhãn nhóm thì không có phương pháp nhóm nào dùng được.

Đằng sau cả bốn bước là một nguyên nhân chung đã nêu ở hộp trực giác: *shortcut learning* [4]. Mô hình không cố hiểu bài toán; nó tìm đặc trưng dễ nhất tương quan đủ mạnh với nhãn trong tập huấn luyện. Nếu ảnh của lớp “bò” toàn chụp trên đồng cỏ thì cỏ là đặc trưng rẻ hơn hình dạng con bò rất nhiều; nếu ảnh bệnh lý đến từ một máy chụp và ảnh bình thường từ máy khác, thì dấu hiệu của máy là đặc trưng rẻ nhất trong toàn bộ ảnh. Đây là lời giải thích thống nhất cho hiện tượng “chính xác trên test, ngu ngốc ngoài đời”, và nó nổi ngược về B/05: shortcut hầu như luôn bắt nguồn từ một thiên lệch chọn mẫu ở khâu thu thập, nên nó không phải lỗi của mô hình mà là lỗi được cài vào từ trước khi mô hình tồn tại.

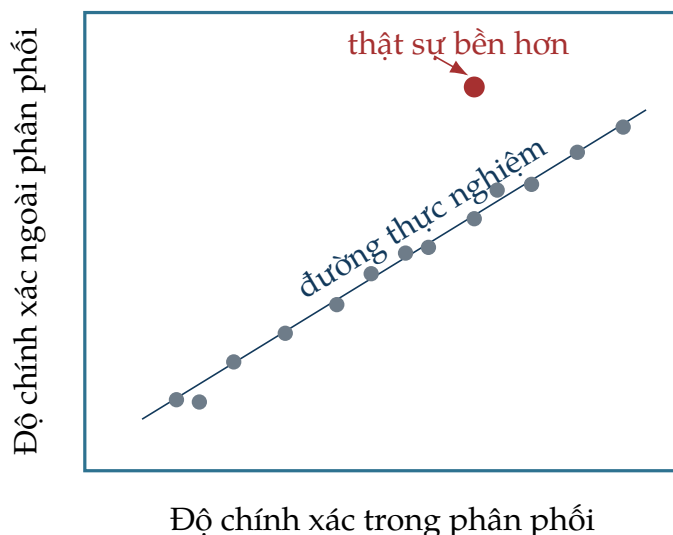
→ hệ quả của B/05 — thiên lệch sinh ở khâu thu thập thì không hàm mất mát nào sửa được ở khâu cuối — shortcut chỉ là cách nó lộ ra.

1.5 Đo độ bền và quyền nói “tôi không biết” (Robustness and OOD Detection)

A. Benchmark corruption, và đường thẳng làm chuẩn nghiêm khắc

Nếu độ bền quan trọng thì nó phải được đo, không phải được hy vọng. Cách rẻ nhất là các bộ benchmark corruption tổng hợp: làm nhiễu, mờ, nén, đổi thời tiết ở nhiều mức nghiêm trọng rồi đọc metric theo từng loại và từng mức [5]. Giá trị thật của chúng nằm ở *cấu trúc* chứ không ở con số tổng — loại corruption làm sập mạnh nhất chính là loại phá huỷ đặc trưng mô hình đang dùng. Nếu mờ chuyển động làm sập trong khi đổi màu không ảnh hưởng, mô hình đang sống nhờ cấu trúc tần số cao, và đó là thông tin hành động được.

Giới hạn thì cũng rõ: corruption tổng hợp không thay thế được dịch chuyển thật. Vì thế các tập dữ liệu như WILDS được xây từ dịch chuyển xảy ra tự nhiên trong ứng dụng: bệnh viện khác, năm khác, vùng địa lý khác [6]. Và có một quan sát thực nghiệm hữu ích, hơi bất ngờ, được vẽ ở Hình 1.2: trên nhiều họ dịch chuyển tự nhiên, độ chính xác ngoài phân phối gần như là hàm tuyến tính của độ chính xác trong phân phối [7], [8].



Hình 1.2. “Accuracy on the line” (sơ đồ minh họa theo mô tả trong [7], không phải số đo lại). Hầu hết mô hình và hầu hết “phương pháp tăng độ bền” chỉ trượt dọc theo một đường thẳng: cải thiện ngoài phân phối đến hoàn toàn từ cải thiện trong phân phối. Chỉ điểm đỏ — nằm *trên* đường — mới xứng đáng gọi là tăng độ bền. Đường thẳng này vì thế là một *baseline* nghiêm khắc mà mọi tuyên bố về độ bền phải vượt qua.

B. Cho mô hình quyền nói “tôi không biết”

↔ **ràng buộc bởi** E/24 — đường thẳng đó là chuẩn so sánh: một ablation về độ bền chỉ có nghĩa khi nó tách được điểm ra khỏi đường.

Hướng bổ sung, không thay thế: **phát hiện ngoài phân phối** (*out-of-distribution detection*) nhằm nhận ra đầu vào nằm ngoài phân phối huấn luyện để hệ thống chuyển sang chế độ an toàn thay vì trả lời bừa. Ba điều cần biết, theo thứ tự quan trọng:

- **Đường cơ sở đơn giản đến bất ngờ** — dùng chính giá trị softmax cực đại làm điểm số [9]. Đây vừa là tin tốt (rẻ, không cần train lại) vừa là cảnh báo: nó cho thấy phần lớn công trình sau đó chỉ cải thiện tiệm tiến trên một tín hiệu vốn yếu.
- **Giới hạn thực nghiệm** — độ tin cậy của mạng sâu vốn đã kém hiệu chuẩn ngay trong phân phối, và nó xuống cấp thêm đúng dưới dịch chuyển, tức đúng lúc ta cần nó nhất [10].
- **Giới hạn nguyên tắc** — không thể có bộ phát hiện OOD phổ quát: “ngoài phân phối” chỉ có nghĩa khi đã cố định một phân phối tham chiếu, nên mọi bộ phát hiện đều là bộ phát hiện của *một loại* bất thường mà bạn đã ngầm chọn.

1.6 Đuôi dài và vòng phản hồi (Long-Tail and Feedback Loops)

Hai hiện tượng cuối không phải dịch chuyển theo nghĩa thời gian, nhưng chúng phá cùng một giả định.

- **Long-tail khác mất cân bằng lớp ở cơ chế, không chỉ ở mức độ.**
 - **Mất cân bằng lớp** — vài lớp có ít mẫu hơn hẳn; tăng trọng số hay lấy mẫu lại giúp được, vì vấn đề nằm ở hàm mất mát.
 - **Đuôi dài** — *rất nhiều* lớp, mỗi lớp cực hiếm; vấn đề không còn là trọng số mà là không đủ mẫu để học biểu diễn. Tăng trọng số cho một lớp có bốn mẫu chỉ làm mô hình thuộc lòng bốn mẫu đó nhanh hơn.

- **Hướng đi có cơ sở** — tách hai việc vốn bị gộp: học biểu diễn trên toàn bộ dữ liệu (phần đầu phân phối cung cấp tín hiệu), rồi học riêng bộ phân loại với dữ liệu cân bằng lại; hoặc đổi hẳn bài toán sang truy hồi theo độ tương đồng, nơi thêm một lớp mới chỉ là thêm một mẫu tham chiếu.
- **vòng phản hồi (*feedback loop*) là thứ nguy hiểm nhất chương, vì nó vô hình với mọi phép đo ngoại tuyến.**
 - **Cơ chế** — hệ thống quyết định cái gì được nhìn thấy: ảnh nào tới tay người kiểm duyệt, vùng nào được quét kỹ, mẫu nào được gửi đi gắn nhãn. Dữ liệu thu về sau đó đã bị chính mô hình lọc.
 - **Vì sao không đo được** — mô hình tiếp theo học trên dữ liệu ấy sẽ củng cố đúng thiên lệch của mô hình trước. Mọi metric ngoại tuyến vẫn xác nhận nó tốt lên, vì tập đánh giá cũng đi qua cùng cái phễu. Đây là một dạng cụ thể của khoản nợ kỹ thuật ẩn trong hệ thống học máy [11].
 - **Cách phá vòng** — cố tình giữ một luồng dữ liệu không bị mô hình lọc: một tỉ lệ nhỏ mẫu chọn ngẫu nhiên, thu thập bất kể mô hình nói gì.

Từ đó suy ra hành động chủ động cuối cùng, và cũng là hành động rẻ nhất trong cả chương: *tự xây một bộ test đối kháng*. Không phải đối kháng theo nghĩa nhiều tối ưu hoá. Ý ở đây là vài trăm ảnh được chọn có chủ ý để lộ đúng shortcut bạn nghi ngờ: cùng vật thể trên nền khác, cùng cảnh chụp bằng camera khác, đúng lớp hiếm ở điều kiện xấu. Vài trăm ảnh như vậy thường nói nhiều hơn cả một bộ benchmark corruption, vì chúng nhắm vào giả thuyết cụ thể của bạn thay vì rải đều.

↔ **đối lập với** B/07 — tập test chuẩn được thiết kế để *đại diện*; bộ test đối kháng được thiết kế để *phá*, nên hai thứ trả lời hai câu hỏi khác nhau.

1.7 Giới hạn và trường hợp hỏng

Cách phân loại ba dịch chuyển ở đầu chương gọn gàng hơn thực tế: dịch chuyển thật hầu như luôn là hỗn hợp của cả ba, và không có phép kiểm định nào tách chúng ra sạch sẽ khi bạn không có nhãn ở miền đích. Thứ tự chẩn đoán thực dụng là bắt đầu bằng cái rẻ nhất — kiểm tra tiên nghiệm lớp có đổi không, vì nếu có thì hiệu chỉnh gần như miễn phí — rồi mới tới các giả thuyết đắt hơn.

Hiệu chỉnh theo tiên nghiệm giả định $p(x | y)$ bất biến, một giả định thường bị vi phạm âm thầm: khi tỉ lệ lớp đổi vì camera mới, thì ảnh của mỗi lớp cũng đã đổi cùng lúc, và bạn có cả label shift lẫn covariate shift chồng lên nhau. Reweighting theo tỉ số mật độ chỉ hoạt động khi vùng mà phân phối nguồn thật sự có mẫu của phân phối đích nằm trong giá đỡ của phân phối nguồn; với miền thật sự mới, tỉ số mật độ tiến ra vô cực ở đúng vùng bạn quan tâm và phương sai của ước lượng nổ tung.

CẠM BÃY

Cạm bẫy hay gặp nhất là dùng chính độ tin cậy của mô hình làm cảnh báo dịch chuyển. Nghe hợp lý — mô hình gặp thứ lạ thì phải kém tự tin — nhưng đó chính là điều mạng sâu *không* làm: dưới dịch chuyển, nó thường vẫn tự tin và tự tin sai [10]. Một hệ thống giám sát chỉ báo động khi confidence tụt sẽ im lặng đúng trong những kịch bản hỏng nguy hiểm nhất.

CẦN NHỚ

“Bị domain shift” không phải một chẩn đoán. Chẩn đoán là trả lời được $p(x)$ đổi, $p(y)$ đổi, hay $p(y | x)$ đổi. Ba câu trả lời đó dẫn tới ba hành động loại trừ nhau, và chỉ cái cuối cùng bắt buộc phải gán nhãn lại. Muốn chẩn đoán được thì phải có một dòng nhỏ dữ liệu *có nhãn* từ miền đích; không có nó thì mọi thuật toán thích nghi đều là đoán mò. [T2]

1.8 Thực hành

Repo và tài nguyên. [bethgelab/imagecorruptions](#) cùng ImageNet-C và ImageNet-R cho corruption tổng hợp; [RobustBench/robustbench](#) để so với các *baseline* đã được đo lại cẩn thận; [p-lambda/wilds](#) cho dịch chuyển thật thay vì nhân tạo; [Jingkang50/OpenOOD](#) và [kkirchheim/pytorch-ood](#) cho OOD detection; [evidentlyai/evidently](#) và [SeldonIO/alibi-detect](#) để giám sát trôi phân phối trong production. Danh sách chốt tại tháng 9/2026 và tên phương pháp dẫn đầu sẽ cũ trong 6–12 tháng; các tập dữ liệu (WILDS, ImageNet-C) và hạ tầng giám sát bền hơn nhiều so với bảng xếp hạng.

Câu hỏi kiểm tra hiểu. Accuracy trên production tụt 8%: ba phép đo nào phân biệt được covariate shift với concept drift, và phép nào rẻ nhất nên làm trước? Vì sao tăng trọng số cho lớp hiếm không giải quyết được đuôi dài? Vì sao một bộ phát hiện OOD “phổ quát” là khái niệm không nhất quán?

Bài tập phụ. Đo accuracy trên 15 loại corruption ở 5 mức nghiêm trọng, vẽ bản đồ nhiệt, tìm loại làm sập mạnh nhất, rồi giải thích nó bằng ngôn ngữ của chuỗi tạo ảnh ở A/02-vat-ly-tao-anh: corruption đó phá huỷ đặc trưng nào, và điều đó nói gì về thứ mô hình đang thật sự dùng?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Giả định IID	Mỗi mẫu được rút độc lập từ cùng một phân phối, cho cả dữ liệu huấn luyện lẫn dữ liệu tương lai. Mọi bảo đảm lý thuyết của học máy đứng trên nó, nên khi nó vỡ thì các bảo đảm mất hiệu lực chứ không chỉ yếu đi.
••• Covariate shift	$p(x)$ đổi nhưng quan hệ ảnh–nhân giữ nguyên: ảnh trông khác, quy luật vẫn đúng. Chữa bằng thích nghi miền, augmentation, hoặc đánh trọng số lại theo mật độ.
••• Label shift	Tỉ lệ các lớp đổi trong khi ảnh của mỗi lớp vẫn y hệt. Đây là trường hợp duy nhất chữa được gần như miễn phí, bằng cách chỉnh lại tiên nghiệm ở đầu ra.
••• Concept drift	Chính $p(y x)$ đổi: cùng một ảnh nay phải gán nhãn khác. Không có đường tắt — phải gán nhãn lại rồi huấn luyện lại.
•• Vì sao label shift chữa được mà không cần huấn luyện lại?	Vì $p(x y)$ không đổi nên tỉ số độ hợp lý giữ nguyên, chỉ phân tiên nghiệm sai. Nhân tỉ số cực của đầu ra với tỉ lệ giữa hai tiên nghiệm là xong.
•• Mô hình cho 0,60 nhưng tỉ lệ dương thật chỉ 5% — xác suất đúng?	Khoảng 0,073. Tỉ số cực 1,5 nhân với 0,0526 ra 0,079, đổi ngược về xác suất là 7,3%. Một dự đoán “khá chắc” hoá ra chỉ là 7%.
••• Bộ phân biệt miền	Một bộ phân loại nhị phân tách “mẫu train” khỏi “mẫu production”. AUC quanh 0,5 nghĩa là chưa có dịch chuyển đáng kể; AUC 0,95 nghĩa là nhìn ảnh cũng biết nó đến từ đâu.
••• Vì sao phát hiện được dịch chuyển vẫn chưa đủ?	Vì nó trả lời “có đổi không”, không trả lời “có hại không”. Đổi tông màu toàn cục là thay đổi lớn mà vô hại; một thay đổi bé đúng vào đặc trưng mô hình dựa vào lại đủ làm sập nó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Thích nghi miền	Căn chỉnh phân phối đặc trưng, hoặc tự huấn luyện bằng nhân giả, khi bạn đã có mẫu từ miền đích. Nó chuyên biệt hoá mô hình cho một phân phối, nên miền thứ ba xuất hiện thì phải làm lại.
●● Tổng quát hoá miền	Học biểu diễn bất biến qua nhiều miền huấn luyện, dùng khi chưa từng thấy miền đích. Giá phải trả là hy sinh hiệu năng trung bình để lấy độ bền.
●● Vì sao tổng quát hoá miền không thắng nổi ERM?	Vì phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình, không từ thuật toán. Cho mọi phương pháp cùng ngân sách tune và cùng thủ tục chọn thì lợi thế biến mất.
●●● Shortcut learning	Mô hình học đặc trưng dễ nhất mà tương quan đủ mạnh với nhãn: cổ thay cho hình dạng con bò, dấu hiệu máy chụp thay cho bệnh lý. Đây là lời giải thích thống nhất cho “chính xác trên test, ngu ngốc ngoài đời”.
●● Accuracy on the line	Quan sát rằng trên nhiều dịch chuyển tự nhiên, độ chính xác ngoài phân phối gần như tuyến tính theo độ chính xác trong phân phối. Dùng nó làm chuẩn: chỉ điểm nằm trên đường mới đáng gọi là tăng độ bền.
●● Benchmark corruption	Đo lại metric qua nhiều loại nhiễu, mờ, nén, thời tiết ở nhiều mức. Giá trị nằm ở bản đồ nhiệt theo từng loại, vì loại làm sập mạnh nhất chính là loại phá đặc trưng mô hình đang dùng.
●●● OOD detection	Nhận ra đầu vào nằm ngoài phân phối huấn luyện để hệ thống nói “tôi không chắc”. Không thể có bộ phát hiện phổ quát, vì “ngoài phân phối” chỉ có nghĩa khi đã cố định một phân phối tham chiếu.
●●● Vì sao độ tin cậy của mô hình là cảnh báo tệ?	Vì mạng sâu vốn đã quá tự tin ngay trong phân phối, và xuống cấp thêm đúng dưới dịch chuyển. Hệ giám sát chỉ báo động khi độ tin cậy tụt sẽ im lặng đúng lúc nguy hiểm nhất.
●●● Đuôi dài	Rất nhiều lớp mà mỗi lớp chỉ có dăm bảy mẫu. Khác mất cân bằng lớp ở chỗ nút thắt là thiếu dữ liệu để học biểu diễn, nên tăng trọng số chỉ làm mô hình thuộc lòng vài mẫu đó nhanh hơn.
●● Vòng phản hồi	Mô hình quyết định cái gì được nhìn thấy, nên dữ liệu thu về sau đó đã bị chính nó lọc. Phá vòng bằng cách giữ một tỉ lệ nhỏ mẫu chọn ngẫu nhiên, thu bắt kể mô hình nói gì.
●● Accuracy production tụt 8% — ba phép đo nào, làm cái nào trước?	Đo tỉ lệ lớp, đo AUC của bộ phân loại phân biệt miền, và gán tay vài chục mẫu để xem nhãn cũ còn đúng không. Làm phép đầu trước vì nếu là label shift thì chữa gần như miễn phí.
●● Mọi metric ngoại tuyến đều tốt lên mà người dùng vẫn phàn nàn — vì sao?	Vòng phản hồi: tập đánh giá đi qua đúng cái phễu mà mô hình đã lọc. Phân biệt bằng một tập giữ riêng thu ngẫu nhiên, không qua mô hình; nếu con số ở đó không nhích thì vòng lặp đã khoá.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
WILDS (p-lambda/wilds)	Đóng gói dịch chuyển tự nhiên theo bệnh viện, theo năm, theo vùng, và yêu cầu báo cáo hiệu năng nhóm tẻ nhất thay vì trung bình	Bằng chứng rằng phân biệt các loại dịch chuyển đã thành chuẩn thiết kế benchmark
ImageNet-C, ImageNet-R, RobustBench	Đo độ bền theo từng loại corruption và từng mức, đúng ý “cấu trúc quan trọng hơn con số tổng”	Hạ tầng đánh giá; RobustBench tồn tại vì các báo cáo tự đo không tái lập được

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenOOD	So các phương pháp phát hiện ngoài phân phối trên cùng giao thức, kể cả <i>baseline</i> softmax cực đại	Cho thấy khoảng cách giữa baseline đơn giản và phương pháp mới hẹp hơn ấn tượng từ từng bài báo
evidentlyai/evidently, SeldonIO/alibi-detect	Giám sát production bằng đúng hai công cụ của chương: khoảng cách phân phối trên embedding và bộ phân loại phân biệt miền	Hạ tầng vận hành, ít phụ thuộc mô hình cụ thể
Mô hình nền tảng huấn luyện trên dữ liệu web quy mô lớn (dòng CLIP, DINOv2/v3)	Được cho là nhắc hẳn đường “accuracy on the line” nhờ đa dạng dữ liệu, chứ không nhờ một thuật toán bền hơn	Đây là cách đọc phổ biến tính tới 9/2026, không phải một kết luận đã khép lại

MINI PROJECT SLAM · Chẩn đoán loại dịch chuyển khi đổi tập dữ liệu · 1–2 ngày

Mục tiêu	lấy đúng bộ phân loại “cùng chỗ” đã dựng ở chương trước, chạy nó trên một tập dữ liệu khác, rồi trả lời bằng bằng chứng số: mức sụt là covariate shift, label shift, hay concept drift.
Dữ liệu	EuRoC làm miền nguồn; TUM-VI (chuỗi room hoặc corridor) hoặc OpenLORIS làm miền đích. Sinh nhân cùng chỗ ở miền đích theo <i>đúng</i> quy tắc hình học đã dùng ở B/07-chia-du-lieu-va-danh-gia, không đổi ngưỡng.
Các bước	bốn phép đo, theo thứ tự từ rẻ tới đắt. (1) Đo metric ở cả hai miền, ghi mức sụt. (2) Huấn luyện một bộ phân loại phân biệt miền trên embedding của hai tập và ghi AUC của nó. (3) Đo tỉ lệ cặp dương ở hai miền, áp hiệu chỉnh tiên nghiệm, ghi phần hiệu năng lấy lại được. (4) Gán tay 50 cặp ở miền đích và đếm tỉ lệ bất đồng giữa nhãn hình học và nhãn tay.
Xong khi	bạn kết luận được một loại dịch chuyển kèm ba con số chống lưng: AUC của bộ phân biệt miền, phần trăm hiệu năng lấy lại nhờ hiệu chỉnh tiên nghiệm, và tỉ lệ bất đồng ở bước (4). Tỉ lệ bất đồng cao chính là bằng chứng của concept drift, vì định nghĩa “cùng chỗ” đã đổi theo cấu trúc cảnh.
Nối với	B/07-chia-du-lieu-va-danh-gia cho bộ đo và cách chia; E/25-do-tin-cay-va-van-hanh vì nếu kết luận là concept drift thì bước kế tiếp là thiết kế chế độ hồng an toàn chứ không phải tinh chỉnh thêm.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba câu hỏi: tỉ lệ lớp có đổi không; ảnh có trông khác không; cùng một ảnh nay có phải gán nhãn khác không. Câu đầu — label shift — chưa được gần như miễn phí bằng hiệu chỉnh tiên nghiệm ở đầu ra, nên phải kiểm tra trước tiên vì nó rẻ nhất.
- A2. Vì nó học nền chứ không học con vật: bò gần như luôn đứng trên cỏ, lạc đà trên cát. Đặc trưng nền rẻ hơn hình dạng rất nhiều và tương quan đủ mạnh với nhãn trong tập huấn luyện, nên ERM chọn đúng nó.
- A3. Vì phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình — mỗi bài chọn siêu tham số theo cách thuận lợi cho phương pháp của mình. Cho mọi phương pháp cùng ngân sách tune và cùng thủ tục chọn thì lợi thế đó biến mất [2].
- A4. Vì mạng sâu vốn đã quá tự tin ngay trong phân phối, và độ tin cậy xuống cấp thêm đúng dưới dịch chuyển. Một hệ giám sát chỉ báo động khi confidence tụt sẽ im lặng đúng trong những kịch bản hồng nguy hiểm nhất.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn bán hàng ở khu phố A suốt ba năm, quen tới mức nhìn khách bước vào là đoán được họ sẽ mua gì. Rồi bạn chuyển sang khu phố B và tỉ lệ đoán đúng tụt hẳn. Chỉ có đúng ba lý do, và ba cách chữa khác hẳn nhau.

Một, khách trông khác đi — ăn mặc khác, đến vào giờ khác — nhưng quy luật cũ vẫn đúng: ai vội thì mua nhanh. Cái này chỉ cần thời gian làm quen với kiểu khách mới. Hai, tỉ lệ khách đổi: khu cũ toàn dân văn phòng, khu mới toàn sinh viên. Quy luật không sai một chút nào, chỉ là bạn đang kỳ vọng sai tỉ lệ, nên chỉnh lại kỳ vọng là xong và gần như không tốn gì. Ba, nghĩa của dấu hiệu đã đổi: ở khu cũ, khách đứng lâu trước quầy nghĩa là sắp mua; ở khu mới, đứng lâu nghĩa là đang đợi bạn. Chỉ trường hợp thứ ba mới bắt bạn quan sát lại từ đầu.

Mẹo là đừng gộp ba thứ đó thành một câu than “khu này khó bán”. Hỏi lần lượt: khách có khác không, tỉ lệ có khác không, dấu hiệu có còn nghĩa cũ không — và hỏi cái rõ nhất trước, vì cái rõ nhất cũng là cái chữa nhanh nhất.

Cái giá là bạn cần vài chục đơn hàng thật ở khu mới mới trả lời được ba câu hỏi ấy. Không có chúng thì bạn chỉ đang đoán, và đoán một cách rất tự tin.

Chỗ không rút gọn được: vì sao máy lại bám vào một dấu hiệu ăn may thay vì dấu hiệu đúng thì không có cách nói gọn nào không gây hiểu lầm. Nói “máy lười” là sai: nó chọn đúng thứ gắn chặt nhất với đáp án trong đồng dữ liệu nó được xem, và nếu chỉ nhìn đồng đó thì không ai — kể cả người — phân biệt được đâu là dấu hiệu thật.

Một câu: *Sự hiệu năng không phải một chẩn đoán; chẩn đoán là nói được cái gì đã đổi, vì mỗi thứ đổi có đúng một cách chữa.*

1.9 Kết nối

Chương này đóng lại Phần B ở phía dữ liệu bằng cách gỡ bỏ giả định mà B/04-nen-tang-hoc-tu-du-lieu phải giả sử và B/07-chia-du-lieu-va-danh-gia phải dựa vào. Nó nối ngược về B/05-thiet-ke-bai-toan qua khái niệm shortcut, vốn chỉ là thiên lệch chọn mẫu nhìn từ phía mô hình, và về B/06-chat-luong-du-lieu theo quan hệ đối lập: ở đó nhấn sai vì con người, ở đây nhấn đúng nhưng thế giới đã đổi. Nó là tiền đề của B/09-thiet-ke-ham-mat-mat, nơi một số dịch chuyển được xử lý ngay trong hàm mất mát, và của E/25-do-tin-cay-va-van-hanh, nơi OOD detection và calibration trở thành một phần của thiết kế chế độ hồng an toàn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một hệ thống có thể có dịch chuyển phân phối rất lớn theo phép đo mà hiệu năng không giảm chút nào, và ngược lại?
2. Bạn đo được tỉ lệ lớp dương đã đổi từ 50% xuống 5%. Vì sao đây là tin tốt, và phép sửa cụ thể gồm mấy bước?
3. Tổng quát hoá miền phải hy sinh cái gì để lấy độ bền, và kết quả của [2] buộc bạn nghi ngờ điều gì trong chính thí nghiệm của mình?
4. Trong Hình 1.2, vì sao một điểm nằm *trên* đường thẳng mới đáng gọi là tăng độ bền, còn một điểm nằm xa về bên phải thì không?
5. Vì sao dùng confidence của mô hình làm cảnh báo dịch chuyển lại im lặng đúng lúc nguy hiểm nhất?
6. Đuôi dài và mất cân bằng lớp khác nhau ở cơ chế nào, và vì sao sự khác nhau đó khiến reweighting thất bại?
7. Một vòng phản hồi làm mọi metric ngoại tuyến tốt lên trong khi hệ thống tụt đi. Cơ chế nào cho phép điều đó, và biện pháp rõ nhất để phá vòng là gì?

Nguồn

- [1] M. Arjovsky, L. Bottou, I. Gulrajani, et al., “Invariant Risk Minimization,” *arXiv:1907.02893*, 2019.
- [2] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” in *ICLR*, 2021.
- [3] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., “Distributionally Robust Neural Networks for Group Shifts,” in *ICLR*, 2020.
- [4] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [5] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [6] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [7] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [8] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [9] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [10] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [11] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.